UDESC

Compiladores

Prof. Ricardo

ricardo.martins@udesc.br

UDESC

Definição Dirigida pela Sintaxe

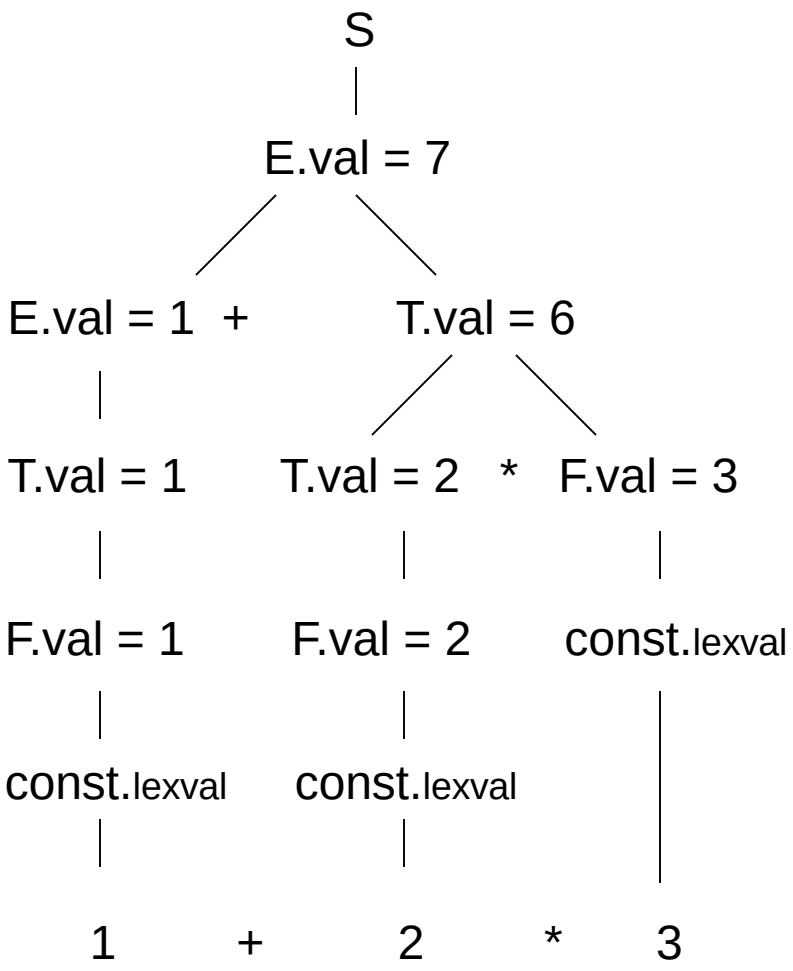
É uma gramática livre de contexto na qual a cada símbolo é associado um conjunto de atributos. A cada produção pode estar associado um conjunto de regras semânticas. Essa regras podem alterar valores de atributos, emitir código, atualizar a tabela de símbolos, emitir mensagens de erro ou realizar quaisquer outras atividades. Em geradores de analisadores sintáticos essas regras são geralmente descritas em uma linguagem de programação.

Os atributos são classificados como:

- **Atributos Sintetizados**: O valor do atributo de um nó é computado a partir dos atributos de seus filhos.
- **Atributos Herdados:** O valor do atributo de um nó é computado a partir dos atributos dos nós irmãos e/ou pais.

Definição Dirigida pela Sintaxe

Produção	Regra Semântica
$S \rightarrow E$	{Imprimir (E.val)}
$E \rightarrow E_1 + T$	{E.val = E_1.val + T.val}
$E \rightarrow T$	{E.val = T.val}
$T \rightarrow T_1 * F$	{T.val = T_1.val * F.val}
$T \rightarrow F$	{T.val = F.val}
$F \rightarrow (E)$	{F.val = E.val}
$F \rightarrow$ ***const***	{F.val = ***const***.*lexval*}



Árvore Sintática Abstrata

$E \rightarrow E_1 + T$ {E.ptr = criarNo (‘+’, E_1.ptr, T.ptr)}

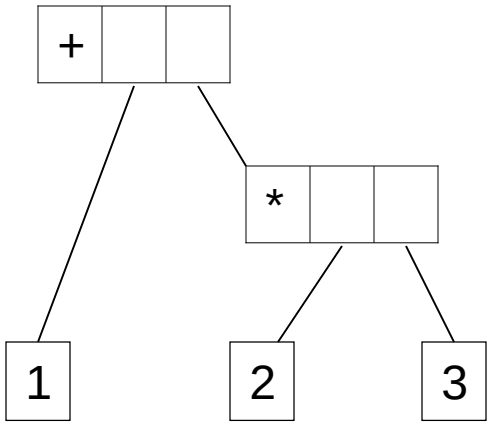
$E \rightarrow T$ {E.ptr = T.ptr}

$T \rightarrow T_1 * F$ {T.ptr = criarNo (‘*’, T_1.ptr, F.ptr)}

$T \rightarrow F$ {T.ptr = F.ptr}

$F \rightarrow (E)$ {F.ptr = E.ptr}

$F \rightarrow$ ***const*** {F.ptr = criarFolha(***const.lexval***)}

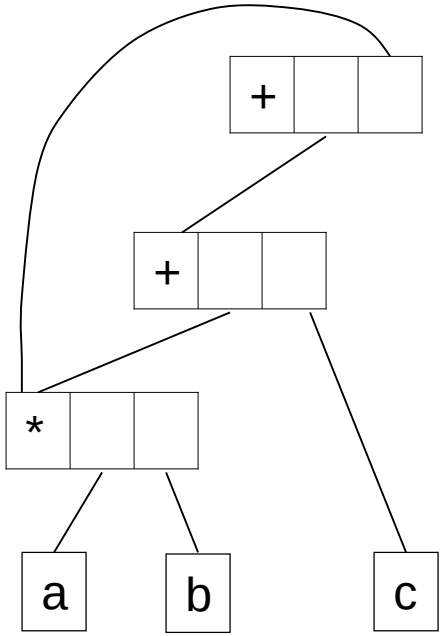


1 + 2 * 3

DAG - Grafo Direcionado Acíclico

Identifica as subexpressões comuns. Exemplo:

DAG que representa a expressão: *a* * *b* + *c* + *a* **b*.



UDESC

Código de 3 endereços

Uma sequência de enunciados na forma:

x = *y* ***op*** *z*

Sendo *x*, *y* e *z* nomes, constantes ou dados temporários (criados pelo compilador) e ***op*** o código que representa uma operação qualquer.

Uma versão linearizada da árvore sintática, é assim chamado por cada instrução poder conter até três endereços, dois para os operandos e um para o resultado. Bastante semelhante a uma linguagem de montagem.

UDESC

Código de 3 endereços

Exemplo: a = 2 * b + c

t1 = 2 * b

t2 = t1 + c

a = t2

Código de 3 endereços

$S \rightarrow$ id = E {S.cod = E.cod ++ gerar(id.lexval = E.local)}

$E \rightarrow E_1$ + T {E.local = novoTemporario();
E.cod = E_1.cod ++T.cod ++ gerar(E.local = E_1.local + T.local)}

$E \rightarrow$ T {E.local = T.local; E.cod = T.cod}

$T \rightarrow T_1$ * F {T.local = novoTemporario();
T.cod = T_1.cod ++ F.cod ++ gerar(T.local = T_1.local * F.local)}

$T \rightarrow$ F {T.local = F.local; T.cod = F.cod}

$F \rightarrow$ (E) {F.local = E.local; F.cod = E.cod}

$F \rightarrow$ id {F.local = id.lexval; F.cod =“”}

$F \rightarrow$ const {F.local = const.lexval; F.cod =“”}

UDESC

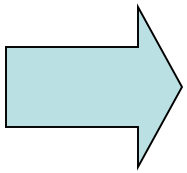
Código de 3 endereços

Alguns enunciados comumente usados:

- x = y op z, onde op é uma operação binária.
- x = op y, onde op é uma operação unária.
- x = y, enunciado de cópia.
- goto L, desvio incondicional.
- if x relop y goto L, onde relop é um operador relacional.
- param x, passagem de parâmetro para funções/procedimentos.
- call p, chamada de uma função/procedimento.
- x = a[i] ou a[i] = x, atribuições indexadas.
- *x = y ou x = *y, indireções.

Código de 3 endereços

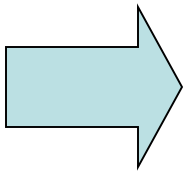
```
n = 1;
f = 1;
while (n < 10)
{
    f = f * n;
    n = n + 1;
}
```



```
    n = 1
    f = 1
L1: if n < 10 goto L2
    goto L3
L2: f = f * n
    n = n + 1
    goto L1:
L3:
```

Código de 3 endereços

```
n = 1;
f = 1;
while (n < 10)
{
    f = f * n;
    n = n + 1;
}
```

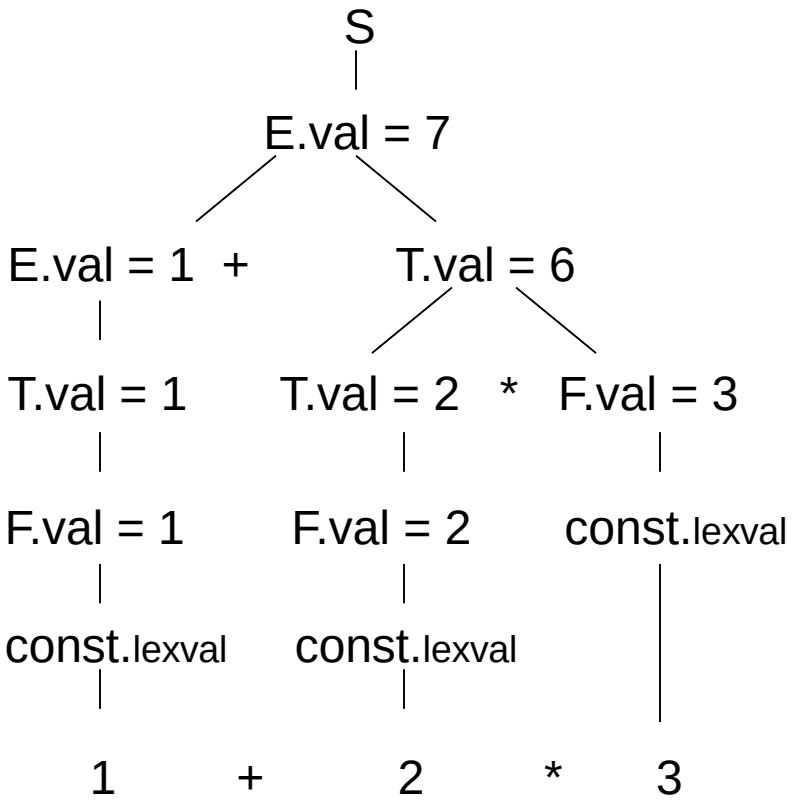


```
    n = 1
    f = 1
L1: if n >= 10 goto L3
    f = f * n
    n = n + 1
    goto L1:
L3:
```

Definições S-atribuídas

Definições dirigidas pela sintaxe que possuem apenas atributos sintetizados.

Produção	Regra Semântica
$S \rightarrow E$	{Imprimir (E.val)}
$E \rightarrow E_1 + T$	{E.val = E_1.val + T.val}
$E \rightarrow T$	{E.val = T.val}
$T \rightarrow T_1 * F$	{T.val = T_1.val * F.val}
$T \rightarrow F$	{T.val = F.val}
$F \rightarrow (E)$	{F.val = E.val}
$F \rightarrow$ ***const***	{F.val = ***const****.lexval*}



UDESC

Definições L-atribuídas

Uma definição dirigida pela sintaxe é *L-atribuída* se cada atributo herdado de X_j, $1 \leq j \leq n$, do lado direito de uma produção, $A \rightarrow X_1X_2...X_n$ depende somente:

1. Dos atributos dos símbolos $X_1X_2...X_{j-1}$ (símbolos a esquerda de X_j).
2. Dos atributos herdados de A.

UDESC

Tradução *Top-Down*

$S \rightarrow E$ {imprimir (E.val)}

$E \rightarrow T$ {E'.h = T.val} E' {E.val = E'.s}

$E' \rightarrow$ **+**T {E'_1.h = E'.h + T.val} E_1 {E'.s = E'_1.s}

$E' \rightarrow \varepsilon$ {E'.s = E'.h}

$T \rightarrow F$ {T'.h = F.val} T' {T.val = T'.s}

$T' \rightarrow$ * F{T'_1.h = T'.h * F.val} T' {T'.s = T_1'.s}

$T' \rightarrow \varepsilon$ {T'.s = T'.h}

$F \rightarrow$ ***const*** {F.val = ***const***.lexval}

$F \rightarrow$**(**{push(T'.h); push(E'.h)} E {E'.h = pop(); T'.h = pop()}**)**

UDESC

Análise Semântica

(Verificação de Contexto)

D $\rightarrow$ **var** S

S $\rightarrow$ **id** L {atribuirTipo(id.lexval, L.tipo)}

S $\rightarrow$ S **id** L {atribuirTipo(id.lexval, L.tipo)}

L $\rightarrow$ **,** **id** L_1 {atribuirTipo(id.lexval, L.tipo); L.tipo = L_1.tipo}

L $\rightarrow$ **:**T {L.tipo = T.tipo}

T $\rightarrow$ **integer** {T.tipo = integer}

T $\rightarrow$ **string** {T.tipo = string}

UDESC

Análise Semântica
(Verificação de Contexto)

$E \rightarrow E_1$ **+** T {if (E_1.tipo = T.tipo) then E.tipo = E_1.tipo else error()}

$E \rightarrow$ T {E.tipo = T.tipo}

$T \rightarrow T_1$ ***** F {if (T_1.tipo = F.tipo) then T.tipo = T_1.tipo else error()}

$T \rightarrow$ F {T.tipo = F.tipo}

$F \rightarrow$ **id** {F.tipo = consultaTipo(id.lexval);}

$F \rightarrow$ **constInt** {F.tipo = Inteiro}

$F \rightarrow$ **constReal** {F.tipo = Real}

Obs: Em uma situação real as regras semânticas devem implementar a coerção dos tipos.

Expressões Lógicas e Relacionais

B $\rightarrow$ B_1 **or** M C {corrigir(B_1.listaf, M.label);
B.listav = merge(B_1.listav, C.listav);
B.listaf = C.listaf;}

B $\rightarrow$ B_1 **and** M C {corrigir(B_1.listav, M.label);
B.listaf = merge(B_1.listaf, C.listaf);
B.listav = C.listav;}

B $\rightarrow$ C {B.listav = C.listav; B.listaf = C.listaf;}

C $\rightarrow$ **not** C_1 {C.listav = C_1.listaf; C.listaf = C_1.listav;}

C $\rightarrow$ **(**B**)** {C.listav = B.listav; C.listaf = B.listaf;}

C $\rightarrow$ E_1 **rel** E_2 {C.listav = criaLista(proxInst);
C.listf = criaLista(proxInst+1);}
gerar(**if** E_1.local **rel** E_2.local **goto** _);
gerar (**goto** _);

M $\rightarrow$ ε {M.label = novolabel()}

Comandos de Seleção e Repetição

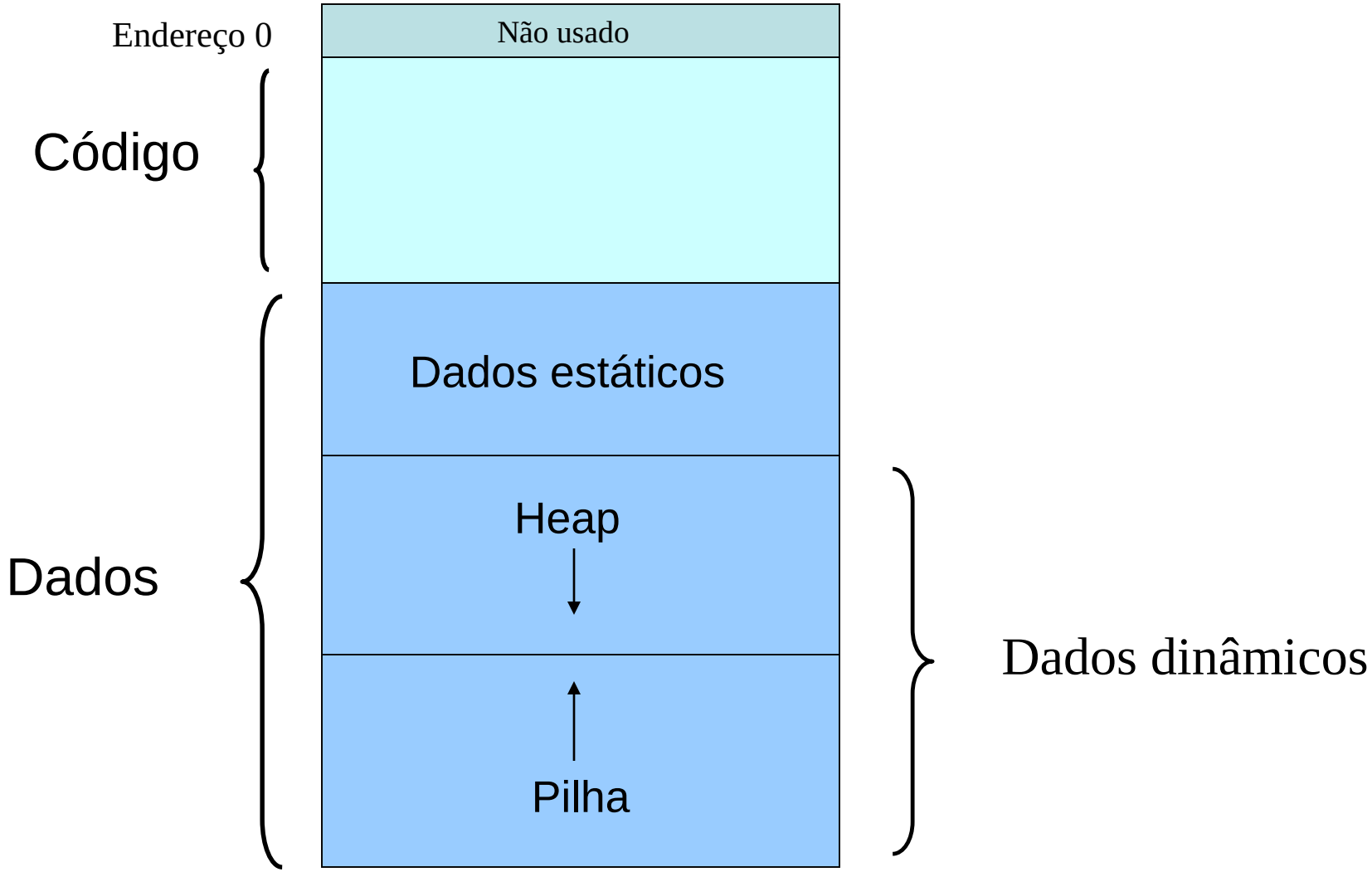
S → **if (**B**) then** M S {corrigir (B.listav, M.label);
corrigir(B.listaf, novolabel();}

S → **if (**B**) then** M_1 S N **else** M_2 S
{corrigir(B.listav, M_1.label);
corrigir(B.listf, M_2.label);
corrigir(N.listav, novoLabel();}

S → **while** M_1 **(**B**)** M_2 S
{corrigir(B.listav, M_2.label);
gerar(goto M_1.label);
corrigir(B.listaf, novolabel();}

N → ε {N.listav = criarLista(ProxInstr); gerar(goto _);}

Organização da Memória



UDESC

Dados Estáticos

A área de memória é reservada no início da execução do programa e liberada apenas no fim de sua execução (*e.g.* variáveis globais e variáveis locais declaradas com o modificador *static* em linguagem C).

UDESC

Dados Dinâmicos

Pilha – Área de armazenamento temporário onde é armazenado o registro de ativação das funções.

Heap – Área reservada para alocação dinâmica, permite ao programador alocar e liberar espaços de memória quando necessário (*e.g.* áreas de memória reservadas pelas funções *malloc* e liberadas pela função *free* em linguagem C).

UDESC

Registro de Ativação

As informações necessárias para execução de uma função/procedimento são gerenciadas utilizando um bloco de memória chamado registro de ativação, fazem parte do registro de ativação: parâmetros, endereço de retorno e variáveis locais.

Chamadas de Funções/Procedimentos

```
int fat (int n)
{
    if (n <= 1)
        return 1;
    return n * fat(n-1);
}
```

```
fat:
    pushl %ebp
    movl %esp, %ebp
    movl 8(%ebp), %ebx
    cmpl $1, %ebx
    jl L1:
    movl $1, %eax
    movl %ebp, %esp
    popl %ebp
    ret
L1: subl $1, %ebx
    pushl %ebx
    call fat
    subl $4, %esp
    movl 8(%ebp), %ebx
    imull %ebx, %eax
    movl %ebp, %esp
    popl %ebp
    ret
```

```
fat:
    pushl %ebp
    movl %esp, %ebp
    movl 8(%ebp), %ebx
    cmpl $1, %ebx
    jl L1:
    movl $1, %eax
    jmp L3
L1: subl $1, %ebx
    pushl %ebx
    call fat
    movl 8(%ebp), %ebx
    imull %ebx, %eax
L3: movl %ebp, %esp
    popl %ebp
    ret
```

Chamadas de Funções/Procedimentos

```
int fat (int n)
{
    if (n >= 1)
        return 1;
    return n * fat(n-1);
}
Int main()
{
    int x;
    x = fat(3);
}
```

